

Análisis y Diseño de Algoritmos

Práctica / Ayudantía

Prof. Gilberto Gutiérrez

Primavera 2025

Problema 1 Sean n y m dos números enteros, con $n \geq m$ y m potencia de 2, es decir, $m = 2^k$ para algún $k = 0, 1, 2, \dots$. Suponga que se requiere implementar la operación de multiplicación $n * m$ por medio de sumas sucesivas. Un algoritmo simple consiste en realizar $O(m)$ sumas.

Mediante la técnica de Dividir para Reiniciar:

1. Diseñe un algoritmo que realice (peor caso) $O(\log_2 m)$ sumas para obtener $n * m$.
2. Plantee la ecuación de recurrencia de su algoritmo que obtiene la cantidad de sumas realizadas en el peor de los casos. Considere que es posible dividir un número por dos, desplazando (shift) un bit a la derecha.
3. Resuelva la ecuación de recurrencia y demuestre que su algoritmo es $O(\log_2 m)$

Problema 2 Considere el algoritmo de ordenamiento MergeSort (visto en clases) el cual divide el arreglo original en dos subarreglos, ordena cada subarreglo de manera recursiva y luego los mezcla. Suponga un nuevo algoritmo, llamado Merge3Sort que divide el arreglo en tres subarreglos, ordena recursivamente cada uno de ellos y luego los mezcla para obtener el arreglo ordenado. Para mezclar los tres subarreglos, existen básicamente dos opciones:

- Primero se mezclan dos de los subarreglos quedando el resultado en un arreglo C . Luego se mezcla el arreglo C con el tercer subarreglo.
- Se mezclan directamente los tres subarreglos. Responda:

1. Plantee las ecuaciones de recurrencia, que modelan el número de comparaciones, en el

peor de los casos, de Merge3Sort. Una para cada forma de realizar la mezcla de los tres subarreglos.

2. Resuelva ambas ecuaciones de recurrencia de Merge3Sort.
3. ¿ Con cuál de las dos formas de mezclar los tres subarreglos de Merge3Sort se obtiene un mejor algoritmo ?
4. Compare el mejor algoritmo de Merge3Sort con MergeSort. ¿Cuál es mejor ?

Problema 3 Sean dos matrices, A y B , de dimensión $n \times n$. La matriz producto $C = A \times B$ también es una matriz de $n \times n$ cuyo elemento (i, j) -ésimo se forma multiplicando cada elemento de la fila i -ésima de A por el elemento correspondiente de la columna j -ésima de B y sumando los productos parciales:

$$C_{ij} = \sum_{k=1}^n A_{ik} \cdot B_{kj}$$

1. **Algoritmo tradicional.** Algoritmo directo, de acuerdo a la definición de multiplicación de matrices.
2. **Algoritmo recursivo (Dividir para Reiniciar).** Suponga ahora que n es potencia de 2; es decir, que existe un entero no negativo k tal $n = 2^k$. (si n no es una potencia de dos pueden añadirse filas y columnas con ceros para que lo sea). Las submatrices A y B pueden dividirse en cuatro submatrices de dimensión $(n/2) \times (n/2)$. Si el producto $A \times B$ tiene la forma:

$$\begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix} \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix} = \begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix}$$

entonces:

$$\begin{aligned}C_{11} &= A_{11} \cdot B_{11} + A_{12} \cdot B_{21} \\C_{12} &= A_{11} \cdot B_{12} + A_{12} \cdot B_{22} \\C_{21} &= A_{21} \cdot B_{11} + A_{22} \cdot B_{21} \\C_{22} &= A_{21} \cdot B_{12} + A_{22} \cdot B_{22}\end{aligned}$$

Para $n = 2$, los elementos de C_{ij} se calculan mediante algunas multiplicaciones y sumas de números, pero para $n > 2$ las submatrices C_{ij} se calculan mediante multiplicaciones (recursivas) y sumas de submatrices de dimensión $(n/2) \times (n/2)$.

3. **Algoritmo de Strassen.** El algoritmo de Strassen calcula las cuatro submatrices C_{ij} empleando sólo 7 multiplicaciones y 18 sumas o restas de matrices. Primero calcula 7 submatrices auxiliares M, N, O, P, Q, R y S , cuya definición es:

$$\begin{aligned}M &= (A_{11} + A_{22}) \cdot (B_{11} + B_{22}) \\N &= (A_{21} + A_{22}) \cdot B_{11} \\O &= A_{11} \cdot (B_{12} - B_{22}) \\P &= A_{22} \cdot (B_{21} - B_{11}) \\Q &= (A_{11} + A_{12}) \cdot B_{22} \\R &= (A_{21} - A_{11}) \cdot (B_{11} + B_{12}) \\S &= (A_{12} - A_{22}) \cdot (B_{21} + B_{22})\end{aligned}$$

posteriormente se calculan las submatrices C_{ij} :

$$\begin{aligned}C_{11} &= M + P - Q + S \\C_{12} &= O + Q \\C_{21} &= N + P \\C_{22} &= M + O - N + R\end{aligned}$$

Se pide:

1. Plantear y resolver las ecuaciones de recurrencia (si corresponde) (número de multiplicaciones para el peor caso) para cada algoritmo (algoritmo tradicional, Dividir para Reinar y de Strassen).
2. Obtener la complejidad de cada algoritmo.
3. De acuerdo a las complejidades de los algoritmos del punto anterior, ¿Cual resultó ser el mejor?
4. Obtenga, al menos dos conclusiones respecto de la aplicación de la técnica de Dividir para Reinar para el problema de la multiplicación de matrices.